

Ecuación del Calor en 2D: Simulación y Resolución Numérica.

SOLUCION TRANSITORIA Y SOLUCIÓN ESTACIONARIA.

Introducción.

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right), \quad \text{con } \alpha = 0.01$$

Condición inicial.

$$T(x, y, 0) = -50 \cos \left(\frac{\pi x}{2} \right) \cos \left(\frac{\pi y}{2} \right)$$

Condiciones de contorno.

Neumann.

$$\frac{\partial T}{\partial x}(0, y, t) = 0, \quad \frac{\partial T}{\partial x}(1, y, t) = 0$$

Dirichlet.

$$T(x, 0, t) = 20, \quad T(x, 1, t) = 50$$

1. Solución transitoria.
 - Método: Esquema implícito en el tiempo.
 - Resolución eficiente mediante matrices dispersas:
 - Formato COO (Coordinate List) para ensamblaje.
 - Conversión a CSR (Compressed Sparse Row) para resolver.
2. Solución estacionaria.
 - Método: Esquema implícito en régimen estático.
 - Eliminación del término temporal.
 - Resolución del sistema lineal con estructura dispersa (CSR manual)

Solución
Transitoria:
Método
Implicito con
Sparse.

01

Solución Transitoria: Método Implícito con Sparse.

$$\frac{\partial T}{\partial t}(\mathbf{x}_i, \mathbf{y}_j, t^n) \approx \frac{T_{i,j}^{n+1} - T_{i,j}^n}{\Delta t}$$

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{i+1,j}^{n+1} - 2T_{i,j}^{n+1} + T_{i-1,j}^{n+1}}{h^2}, \quad \frac{\partial^2 T}{\partial y^2} \approx \frac{T_{i,j+1}^{n+1} - 2T_{i,j}^{n+1} + T_{i,j-1}^{n+1}}{h^2}$$

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right), \quad \text{con } \alpha = 0.01$$

$$\mathbf{A} \cdot \vec{T}^{n+1} = \vec{T}^n \quad \text{con } \mathbf{A} \in \mathbb{R}^{N^2 \times N^2}$$

$$(1 + 4\lambda)T_{i,j}^{n+1} - \lambda T_{i+1,j}^{n+1} - \lambda T_{i-1,j}^{n+1} - \lambda T_{i,j+1}^{n+1} - \lambda T_{i,j-1}^{n+1} = T_{i,j}^n$$

$$\mathbf{k} = \mathbf{i} + \mathbf{j} \cdot \mathbf{N} \quad \text{con } \mathbf{i}, \mathbf{j} = 0, \dots, \mathbf{N} - 1$$

- $A_{k,k} = 1 + 4\lambda$
 $A_{k,k-1} = A_{k,k+1} = -\lambda$
 $A_{k,k-N} = A_{k,k+N} = -\lambda$
- Contribución del nodo central,
Si $k - 1$ y $k + 1$ están en la misma fila,
Si los nodos superior/inferior están dentro del dominio.

$$\begin{bmatrix} 1 + 4\lambda & -\lambda & 0 & \dots & -\lambda & 0 & \dots \\ -\lambda & 1 + 4\lambda & -\lambda & \dots & 0 & -\lambda & \dots \\ 0 & -\lambda & 1 + 4\lambda & \dots & 0 & 0 & \dots \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots \\ -\lambda & 0 & 0 & \dots & 1 + 4\lambda & -\lambda & \dots \\ 0 & -\lambda & 0 & \dots & -\lambda & 1 + 4\lambda & \dots \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots \end{bmatrix}_{N^2 \times N^2} \cdot \begin{bmatrix} T_{0,0}^{n+1} \\ T_{1,0}^{n+1} \\ \vdots \\ T_{N-1,0}^{n+1} \\ T_{0,1}^{n+1} \\ \vdots \\ T_{N-1,N-1}^{n+1} \end{bmatrix} = \begin{bmatrix} T_{0,0}^n \\ T_{1,0}^n \\ \vdots \\ T_{N-1,0}^n \\ T_{0,1}^n \\ \vdots \\ T_{N-1,N-1}^n \end{bmatrix} \quad \text{con } \lambda = \frac{\alpha \Delta t}{h^2}$$

Solución Transitoria: Método Implícito con Sparse.

1. Discretización espacio-temporal: Esquema implícito. Se discretiza con diferencias finitas:
 - a. Resultado: sistema lineal por paso temporal.
2. Construcción de la matriz A:
 - Diagonal principal: $1+4\lambda$
 - Vecinos: $-\lambda$ (siempre que estén dentro del dominio)
 - Tratamiento de condiciones de contorno:
 - Dirichlet: Temperaturas fijas en $y=0$ y $y=1$
→ Imponen directamente en el sistema.
 - Neumann homogénea: derivada nula en $x=0$ y $x=1$ → $T_{N+1,j}^n = T_{N-1,j}^n$ $T_{-1,j}^n = T_{1,j}^n$
3. Conversión a CSR (Compressed Sparse Row).
4. Inicialización de la condición inicial.
5. Bucle en el tiempo y visualización de resultados.

```
# Parámetros físicos.
alpha = 0.01
L = 1.0

# Discretización espacial.
Nx = Ny = 20 # Mallado cuadrado.
dx = L / (Nx - 1) # Espaciado entre nodos.

# Discretización temporal.
dt = 0.01 # Paso de tiempo.
T_max = 5 # Tiempo total de simulación.
Nt = int(T_max / dt) # Número de pasos de tiempo.

# Malla espacial.
x = np.linspace(0, L, Nx) # Vector de Nx puntos espaciados
y = np.linspace(0, L, Ny) # Vector de Ny puntos espaciados
X, Y = np.meshgrid(x, y) # Creación de la malla 2D: Matrices

# Condición inicial.
T = -50 * np.cos(np.pi * X / 2) * np.cos(np.pi * Y / 2) # M

# Simulación y captura de frames.
frames = []
for n in range(Nt): # Iterar sobre el tiempo.
    T_flat = T.flatten() # Convertir a 1D para la matriz dispersa.
    T_flat[:Nx] = 20 # y = 0.
    T_flat[-Nx:] = 50 # y = 1.
    T_new = spsolve(A_sparse, T_flat) # A*x=b.
    T = T_new.reshape((Ny, Nx)) # Volver a 2D.
    if n % 5 == 0: # Guardar cada 5 pasos (~100 frames tot
        frames.append(T.copy())

# Construcción de la matriz A mediante Sparse: Esquema de 5 puntos.
def build_sparse_matrix_A(Nx, Ny, alpha, dt, dx):
    N = Nx * Ny # Número total de nodos: N^2.
    A = lil_matrix((N, N)) # Matriz dispersa vacía: Cada fila se representa como una lista de columnas
    lam = alpha * dt / dx**2 # Número de Fourier: lambda.
    for j in range(Ny):
        for i in range(Nx):
            idx = j * Nx + i # De 2D a 1D.
            if j == 0 or j == Ny - 1: # Dirichlet en y=0 y y=L. Valor fijo.
                A[idx, idx] = 1
                continue
            A[idx, idx] = 1 + 4 * lam # Coeficiente de la diagonal principal.
            if i == 0: # Neumann x=0.
                A[idx, idx + 1] = -2 * lam # Reflexión simétrica.
            elif i == Nx - 1: # Neumann x=L.
                A[idx, idx - 1] = -2 * lam # Reflexión simétrica.
            else:
                A[idx, idx - 1] = -lam # Izquierda.
                A[idx, idx + 1] = -lam # Derecha.
            A[idx, idx - Nx] = -lam # Arriba.
            A[idx, idx + Nx] = -lam # Abajo.
    return csc_matrix(A) # Convertir a formato CSR para eficiencia en la solución.

A_sparse = build_sparse_matrix_A(Nx, Ny, alpha, dt, dx)
```

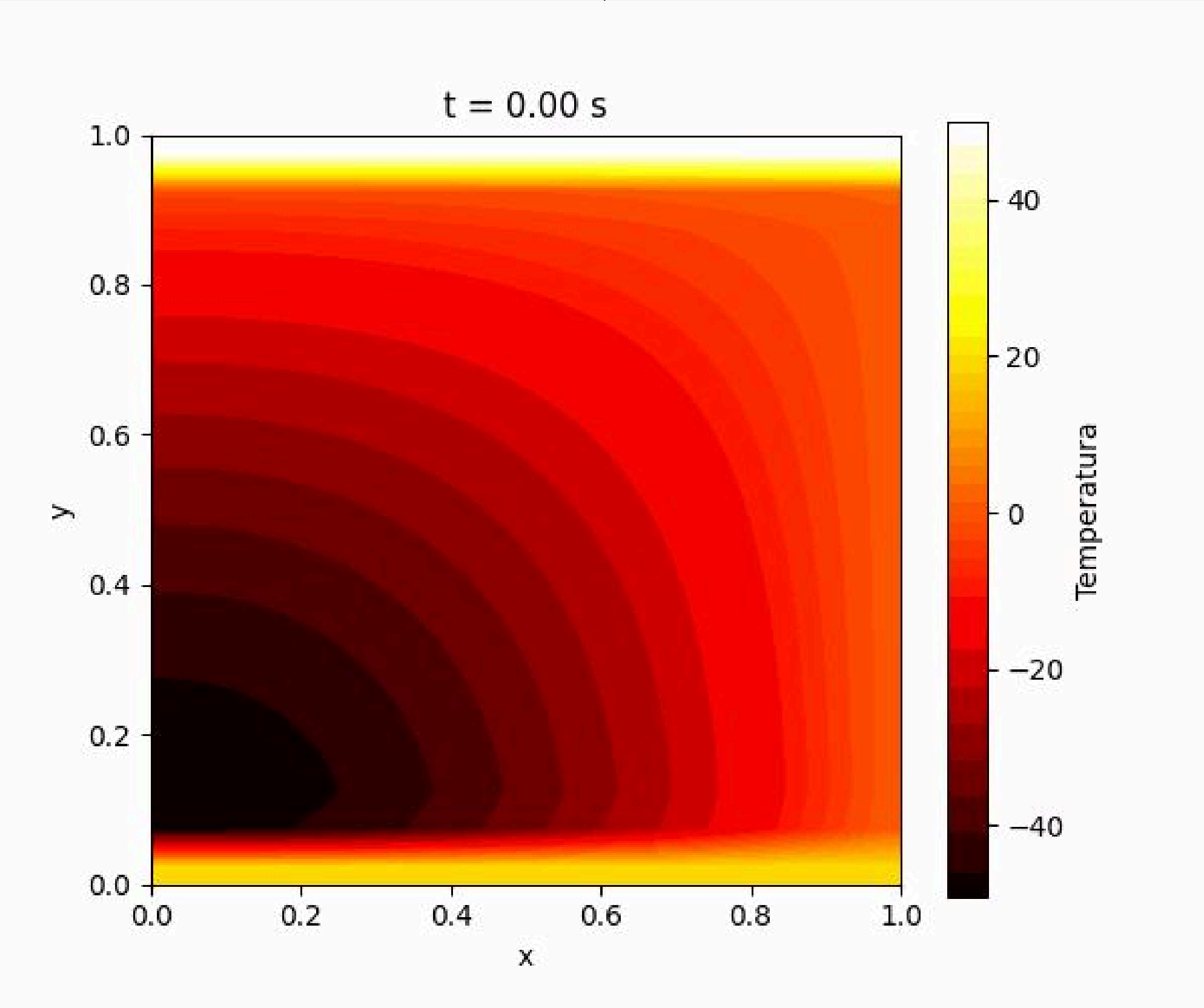
Resultados: Régimen Transitorio.

La simulación comienza con una distribución de temperatura negativa en el dominio, que se redistribuye progresivamente hacia una configuración más homogénea a lo largo del tiempo.

DIFUSIÓN TÉRMICA PROGRESIVA.

INFLUENCIA DE LAS CONDICIONES DE CONTORNO

Los bordes con temperatura fija (Dirichlet) impulsan el flujo térmico, mientras que los bordes aislantes (Neumann) generan simetría horizontal.



Se observa que a partir de los 5 segundos el sistema alcanza un estado prácticamente constante, marcando el inicio del régimen estacionario.

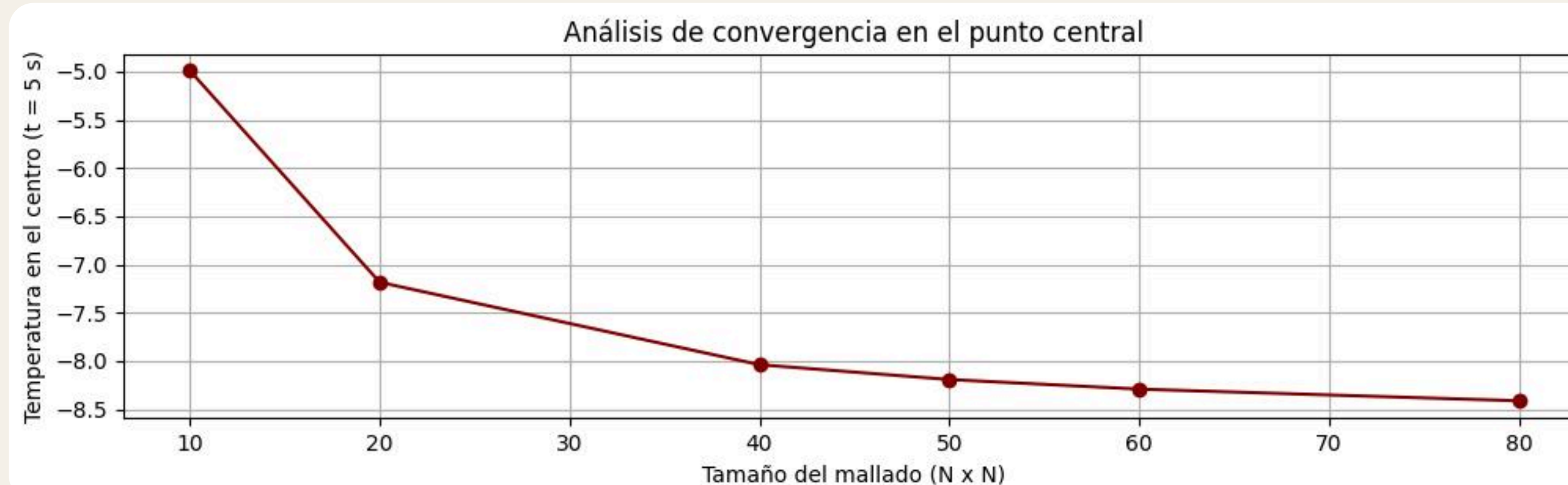
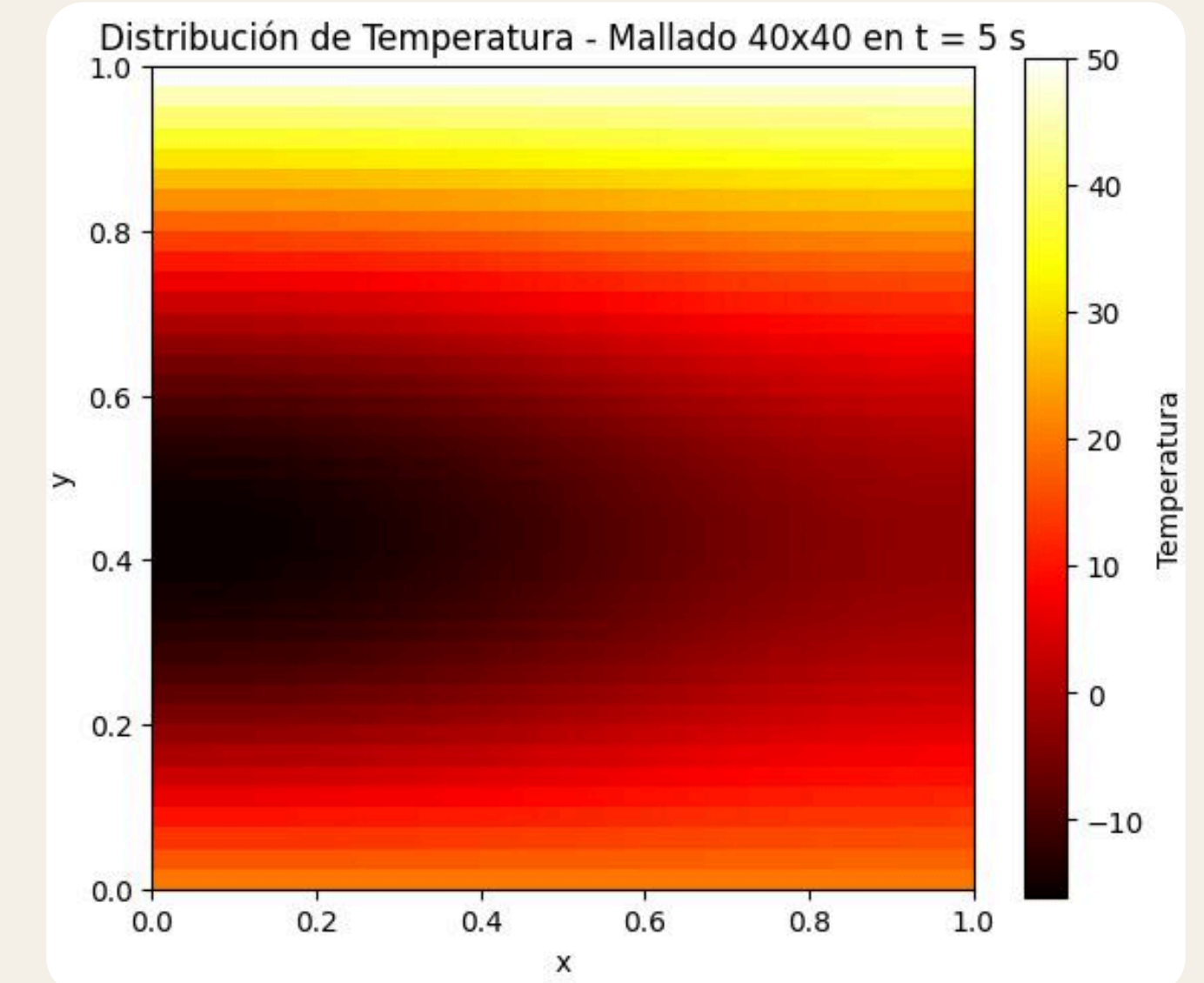
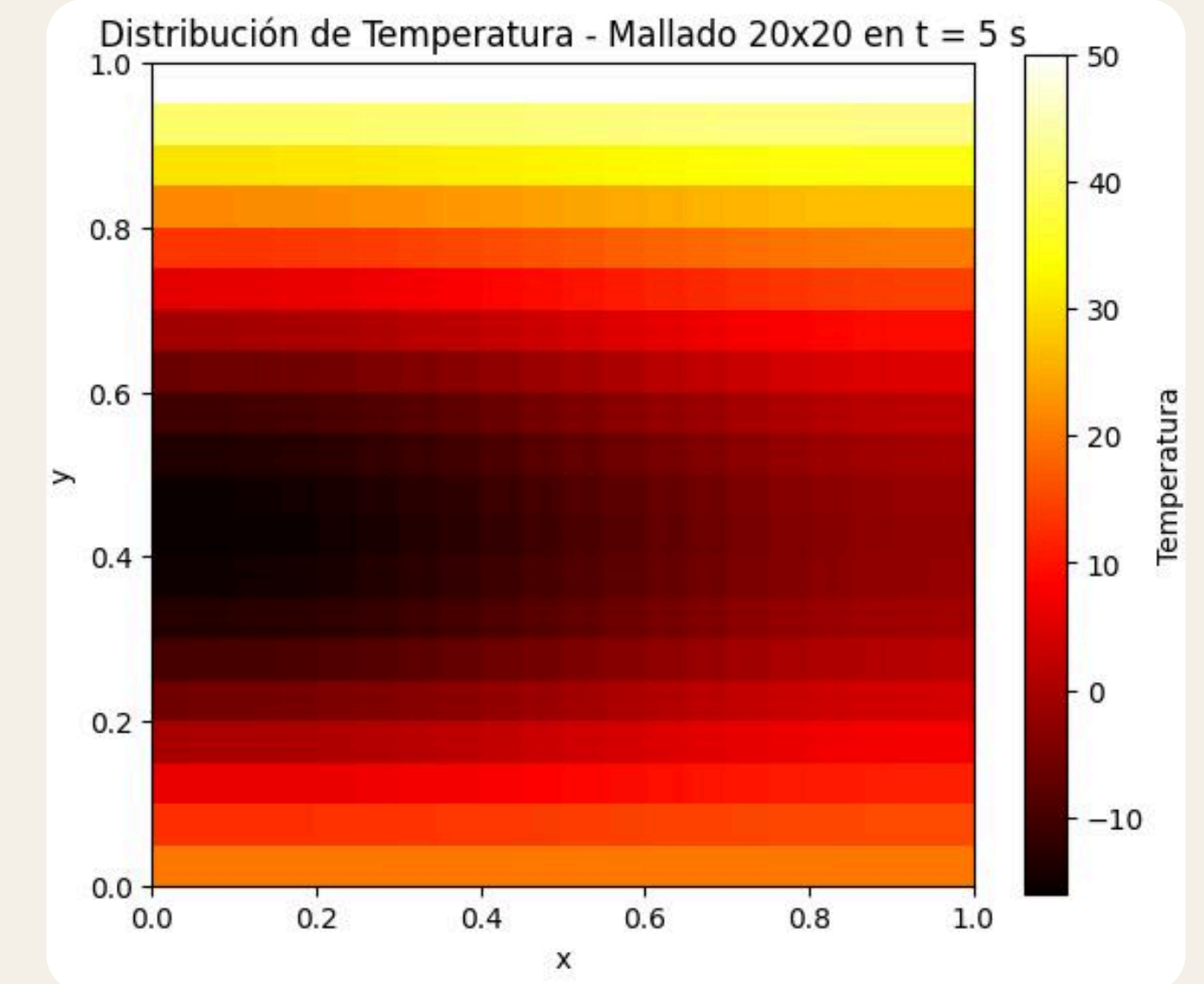
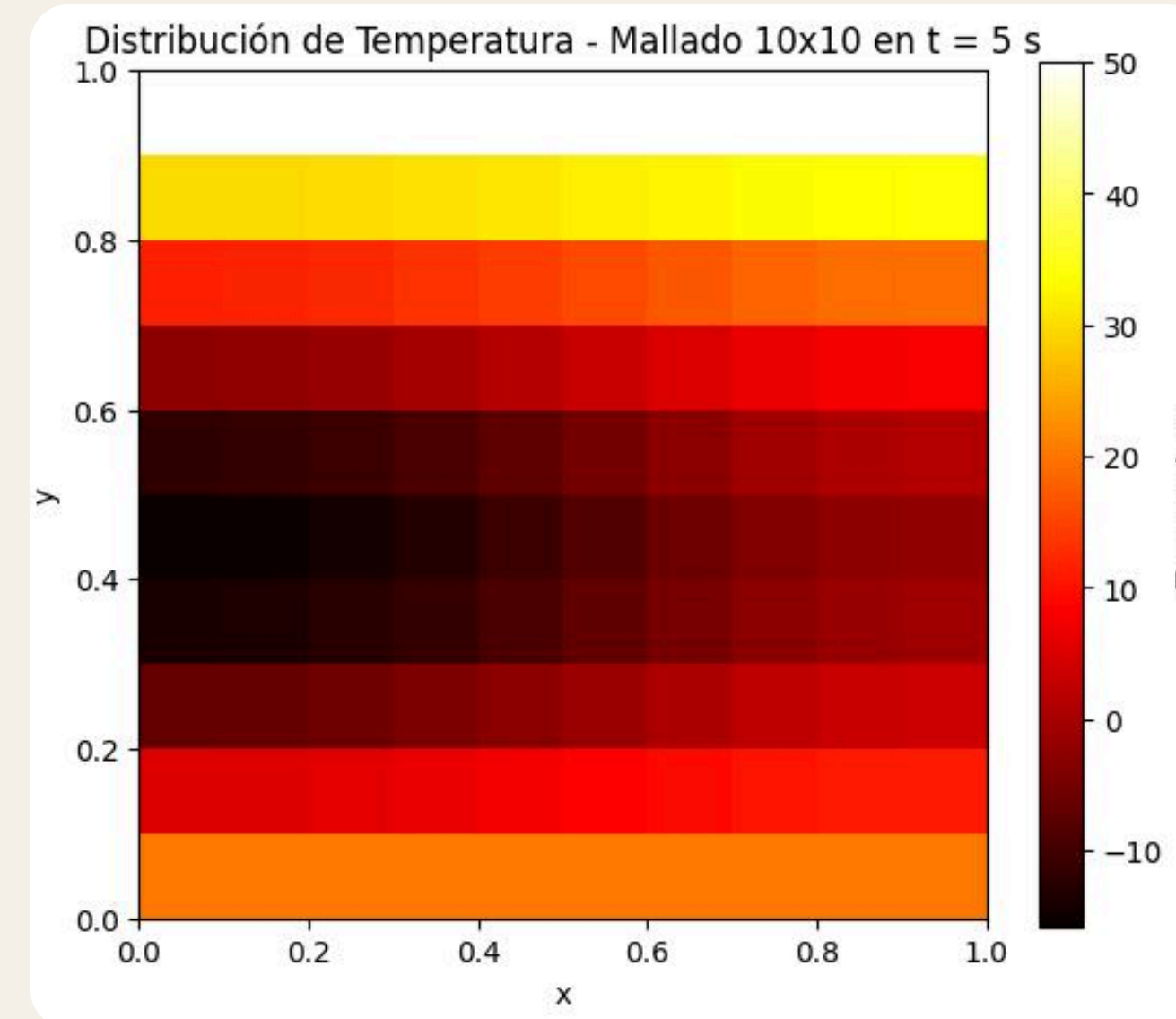
TRANSICIÓN AL EQUILIBRIO

ESTABILIDAD DEL MÉTODO NUMÉRICO

La elección del esquema implícito con matrices dispersas asegura una evolución suave y estable, sin oscilaciones numéricas ni inestabilidades.

Resultados: Comparativa de mallados.

- **Convergencia asegurada:** Las soluciones se aproximan al refinar la malla.
- **Mallado fino vs. grueso:** Más detalle o más velocidad, según el caso.
- **Método eficiente:** Buen rendimiento gracias a matrices dispersas.
- **Balance necesario:** Precisión vs. coste computacional.



Solución Transitoria: Comparación de métodos numéricos.

Criterio	Método Explícito	Método Implícito	Implícito con Sparse
Estabilidad	Condicional (CFL)	Incondicional	Incondicional
Eficiencia computacional	Rápido por paso, ineficiente global	Lento por paso	Rápido y escalable
Memoria y escalabilidad	Limitado a mallas pequeñas	Elevado uso de memoria	Eficiente en memoria (sparse)
Complejidad de implementación	Muy baja	Media	Alta (requiere estructuras dispersas)
Aplicación práctica	Solo casos simples	Casos medios	Casos grandes y realistas

Solución Estacionaria: Método Implicito con Sparse Manual.

O2

Solución Estacionaria: Método Implícito con Sparse Manual.

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{i+1,j}^{n+1} - 2T_{i,j}^{n+1} + T_{i-1,j}^{n+1}}{h^2}, \quad \frac{\partial^2 T}{\partial y^2} \approx \frac{T_{i,j+1}^{n+1} - 2T_{i,j}^{n+1} + T_{i,j-1}^{n+1}}{h^2}$$

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = 0$$

$$\mathbf{A} \cdot \vec{T} = \vec{b} \quad \text{con } \mathbf{A} \in \mathbb{R}^{N^2 \times N^2}$$

$$\mathbf{k} = \mathbf{i} + \mathbf{j} \cdot \mathbf{N} \quad \text{con } \mathbf{i}, \mathbf{j} = 0, \dots, \mathbf{N} - 1$$

$$\begin{aligned} A_{k,k} &= -4 \\ A_{k,k-1} &= A_{k,k+1} = 1 \\ A_{k,k-N} &= A_{k,k+N} = 1 \end{aligned}$$

Contribución del nodo central,
Si $k - 1$ y $k + 1$ están en la misma fila,
Si los nodos superior/inferior están dentro del dominio.

$$\begin{bmatrix} 4 & -1 & 0 & \dots & -1 & 0 & \dots & 0 \\ -1 & 4 & -1 & \dots & 0 & -1 & \dots & 0 \\ 0 & -1 & 4 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ -1 & 0 & 0 & \dots & 4 & -1 & \dots & 0 \\ 0 & -1 & 0 & \dots & -1 & 4 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & -1 \\ 0 & 0 & 0 & \dots & 0 & -1 & -1 & 4 \end{bmatrix}_{N^2 \times N^2}$$

$$\cdot \begin{bmatrix} T_{0,0} \\ T_{1,0} \\ \vdots \\ T_{N-1,0} \\ T_{0,1} \\ \vdots \\ T_{N-1,N-1} \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_{N^2-1} \end{bmatrix}$$

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

Matrices Sparse: COO y CSR.

$$\mathbf{A} \cdot \vec{T} = \vec{b} \quad \text{con } \mathbf{A} \in \mathbb{R}^{N^2 \times N^2}$$



USO DE MATRICES DISPERSAS

1. **La matriz del sistema es muy dispersa:** solo hay hasta 5 valores no nulos por fila.
2. Almacenar ceros sería ineficiente en memoria y cálculo, especialmente en mallas grandes.



FORMATO COO (Coordinate List)

Representa solo los valores no nulos mediante tres vectores:

- **data:** valores numéricos
- **row:** índice de fila
- **col:** índice de columna

Ideal para ensamblar la matriz fácilmente, fila a fila.



FORMATO CSR (Compressed Sparse Row)

Usa:

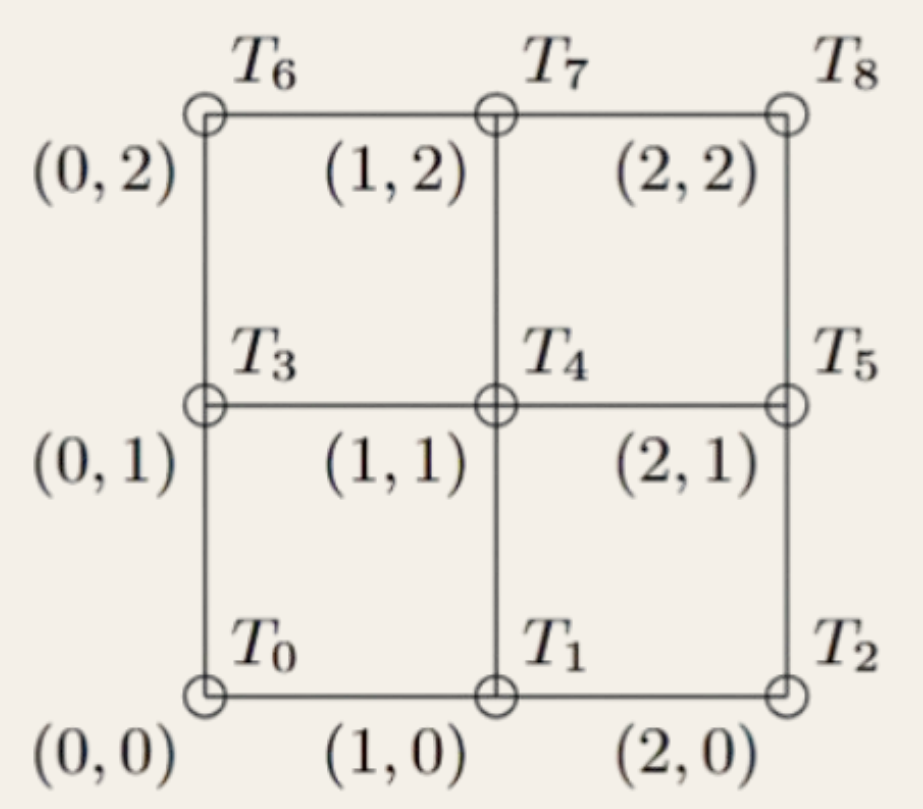
- **data:** valores no nulos ordenados por fila
- **col_ind:** columna de cada valor
- **row_ptr:** inicio de cada fila en data

Permite acceso eficiente por filas → óptimo para resolver el sistema.

Matrices Sparse: COO y CSR – Ejemplo 3×3.

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

$$\mathbf{A} \cdot \vec{T} = \vec{b} \quad \text{con } \mathbf{A} \in \mathbb{R}^{N^2 \times N^2}$$



Construcción COO.

Ejemplo: nodo 0 (esquina inferior izquierda)
→ Vecinos útiles: nodo 1, nodo 3.
→ Contorno (abajo) ya incorporado en b.

row+ = [0, 0, 0]
col+ = [0, 1, 3]
data+ = [-4, 2, 1]

Ejemplo: nodo 4 (centro del dominio)
• Vecinos: 1, 3, 5, 7 + sí mismo

row+ = [4, 4, 4, 4, 4]
col+ = [1, 3, 5, 7, 4]
data+ = [1, 1, 1, 1, -4]

$$\mathbf{A} =$$

$$\begin{bmatrix} -4 & 2 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -4 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 2 & -4 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & -4 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & -4 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & -4 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & -4 & 2 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & -4 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 2 & -4 \end{bmatrix}$$

Construcción CSR.

A partir del ensamblado en COO, obtenemos:

data → valores no nulos de la matriz:
[-4, 2, 1, ..., 1, 1, 1, 1, -4, ...]
col_ind → columnas asociadas a cada valor en data:
[0, 1, 3, ..., 1, 3, 5, 7, 4, ...]
row_ptr → marcas del inicio de cada fila en data:
[0, 3, 7, 10, 14, 19, 23, 26, 30, 33]

El vector row_ptr nos dice en qué posiciones de data y col_ind comienzan y terminan los elementos de cada fila de la matriz.

Matrices Sparse: Resolución Gauss-Seidel.

1 – SISTEMA LINEAL

Queremos encontrar una solución $\mathbf{x} \in \mathbb{R}^n$ tal que:

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \qquad \sum_{j=0}^{n-1} \mathbf{A}_{ij} \mathbf{x}_j = \mathbf{b}_i$$

Cada fila del sistema representa una ecuación lineal.

2 – DESPEJE INCÓGNITA

Despejamos x_i en la ecuación de la fila i . Base del método iterativo.

$$x_i = \frac{1}{A_{ii}} \left(b_i - \sum_{j \neq i} A_{ij} x_j \right)$$

3 – ITERACIÓN GAUSS SEIDEL

Esto hace que el método aproveche al máximo las actualizaciones, mejorando su convergencia.

$$x_i^{(k+1)} = \frac{1}{A_{ii}} \left(b_i - \sum_{j < i} A_{ij} x_j^{(k+1)} - \sum_{j > i} A_{ij} x_j^{(k)} \right)$$

4 – APLICACIÓN CSR

1. Para cada fila i , recorremos desde:
`row_ptr[i] hasta row_ptr[i+1] - 1`

2. Localizamos A_{ii} ;
`col_ind[j] = i`

3. Para los demás j , acumulamos:
Si `col_ind[j] < i` : usar $x_j^{(k+1)}$
Si `col_ind[j] > i` : usar $x_j^{(k)}$

4. Finalmente: $x_i^{(k+1)} = \frac{b_i - \text{suma}}{A_{ii}}$

5 – CONVERGENCIA

Comprobamos si: $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_\infty < \varepsilon$

Los valores más recientes $x_j^{(k+1)}$ se usan para $j < i$
Los antiguos $x_j^{(k)}$ se usan para $j > i$

Solución Estacionaria: Método Implícito con Sparse.

1. Dominio y condiciones de contorno.
2. Construcción de la matriz en formato CSR.
3. Ensamblado nodo a nodo:
 - a. Condiciones de contorno (Dirichlet): Si $j=0$ o $j=N_y-1$:
 - i. Se fija el valor en $b[idx]$
 - ii. Se pone 1 en la diagonal.
 - b. Nodo interior – esquema de 5 puntos:
 - i. Centro: -4
 - ii. Vecinos: $+1$
 - c. Bordes laterales – Neumann homogénea:
 - i. Reflexión $\rightarrow$ duplicación del vecino
 - Borde izquierdo: $+2 \cdot T_{i+1,j}$
 - Borde derecho: $+2 \cdot T_{i-1,j}$
 - d. Conexiones verticales: $+1$ para $T_{i,j-1}$ y $T_{i,j+1}$
4. Gauss–Seidel en CSR.

```
# Parámetros inicialización.
L = 1.0 # Longitud del dominio.
Nx = Ny = 20 # Mallado cuadrado.
dx = L / (Nx - 1) # Espaciado entre nodos.
N = Nx * Ny # Número total de nodos.

# Condiciones de contorno.
T_abajo = 20 # Temperatura en y=0.
T_arriba = 50 # Temperatura en y=L.
b = np.zeros(N) # Vector b para el sistema Ax=b.

# Método de Gauss-Seidel para resolver Ax=b.
def gauss_seidel_csr(data, indices, indptr, b, tol=1e-5, max_iter=10000):
    # Inicializar variables.
    N = len(b)
    x = np.zeros_like(b)

    for _ in range(max_iter):
        x_old = x.copy() # Guardar la solución anterior para la convergencia.
        for i in range(N):
            row_start = indptr[i] # Inicio de la fila i.
            row_end = indptr[i + 1] # Fin de la fila i.
            sum_ = 0
            diag = 0
            for k in range(row_start, row_end): # Recorrer la fila i.
                col = indices[k] # Columna correspondiente al dato.
                val = data[k] # Valor del dato.
                if col == i: # Diagonal principal.
                    diag = val
                else: # Sumar los productos de la fila.
                    sum_ += val * x[col]
            x[i] = (b[i] - sum_) / diag # Actualizar la solución.
            if np.linalg.norm(x - x_old, ord=np.inf) < tol:
                break
    return x

T_flat = gauss_seidel_csr(data, indices, indptr, b)
T = T_flat.reshape((Ny, Nx)) # Volver a 2D.

# Construcción de la matriz A en formato CSR.
data = [] # Datos no nulos de la matriz.
indices = [] # Índices de las columnas correspondientes a los datos.
indptr = [0] # Puntero para la posición de inicio de cada fila.

for j in range(Ny):
    for i in range(Nx): # Recorrer cada nodo de la malla.
        idx = j * Nx + i # De 2D a 1D.
        row_data = [] # Datos de la fila actual.
        row_indices = [] # Índices de la fila actual.

        if j == 0: # Dirichlet en y=0.
            row_data.append(1)
            row_indices.append(idx)
            b[idx] = T_abajo

        elif j == Ny - 1: # Dirichlet en y=L.
            row_data.append(1)
            row_indices.append(idx)
            b[idx] = T_arriba

        else:
            # Diagonal central.
            row_data.append(-4)
            row_indices.append(idx)

            if i == 0: # Neumann en x=0.
                row_data.append(2)
                row_indices.append(idx + 1)
            elif i == Nx - 1: # Neumann en x=L.
                row_data.append(2)
                row_indices.append(idx - 1)
            else: # Interior del dominio.
                row_data += [1, 1]
                row_indices += [idx - 1, idx + 1]

            # Arriba y abajo.
            row_data += [1, 1]
            row_indices += [idx - Nx, idx + Nx]

        # Añadir a CSR.
        data.extend(row_data)
        indices.extend(row_indices)
        indptr.append(len(data))
```

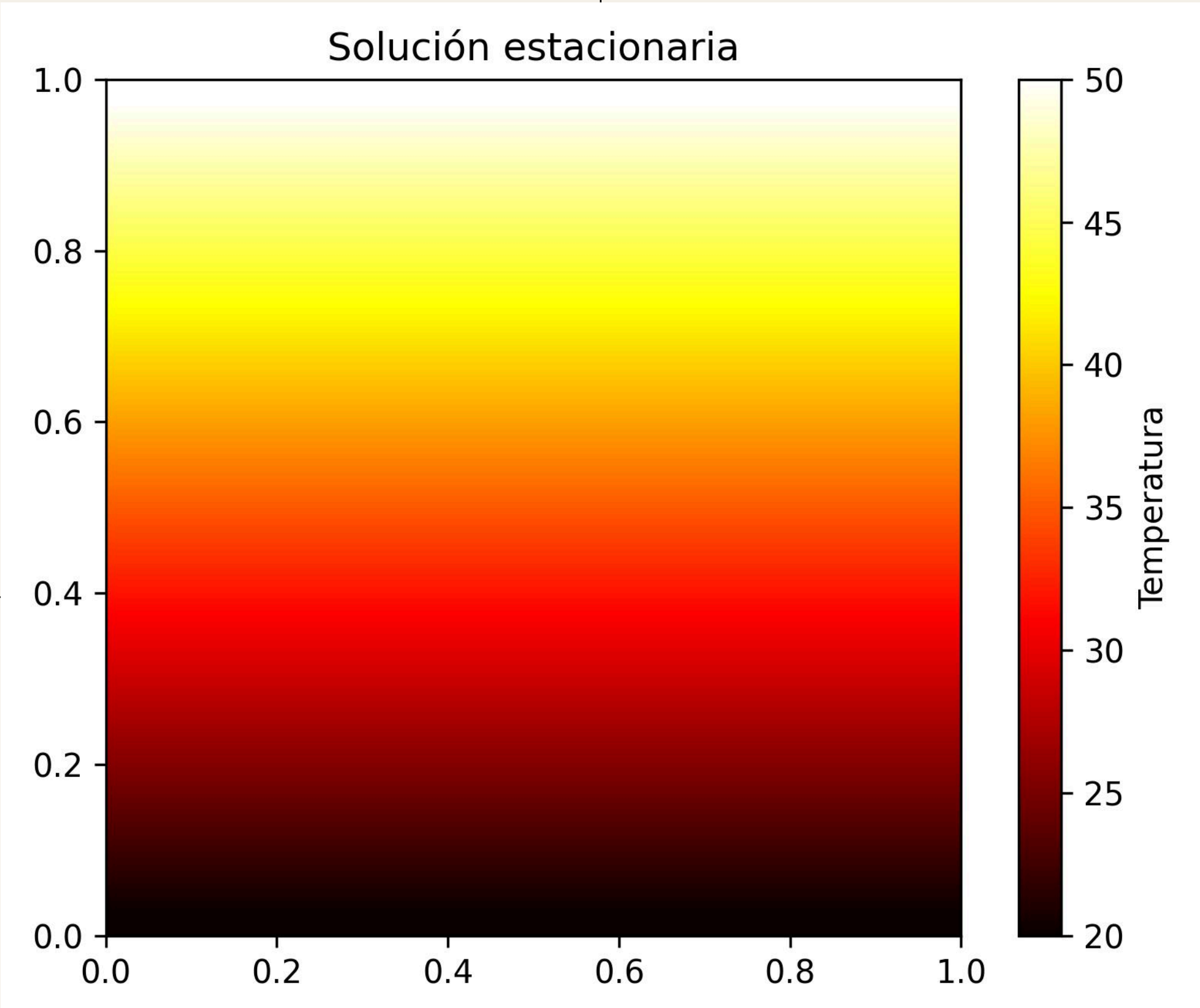
Resultados: Régimen Estacionario.

La solución estacionaria muestra una distribución térmica suave y simétrica, resultado del equilibrio impuesto por los bordes calientes y fríos.

DIFUSIÓN TÉRMICA

INFLUENCIA DE LAS CONDICIONES DE CONTORNO

- Borde superior (Dirichlet): temperatura fija de 50°C.
- Borde inferior (Dirichlet): temperatura fija de 20°C.
- Bordes izquierdo y derecho (Neumann): sin flujo de calor.



La solución estacionaria representa el estado final hacia el que converge el sistema transitorio, confirmando que se ha alcanzado el equilibrio térmico bajo las condiciones de contorno dadas.

TRANSICIÓN AL EQUILIBRIO

ESTABILIDAD DEL MÉTODO NUMÉRICO

El uso del método explícito reformulado con matrices sparse permite una solución estable y eficiente, sin oscilaciones ni errores numéricos, incluso en mallas más densas.

Resultados: Comparativa de mallados.

Convergencia Visual

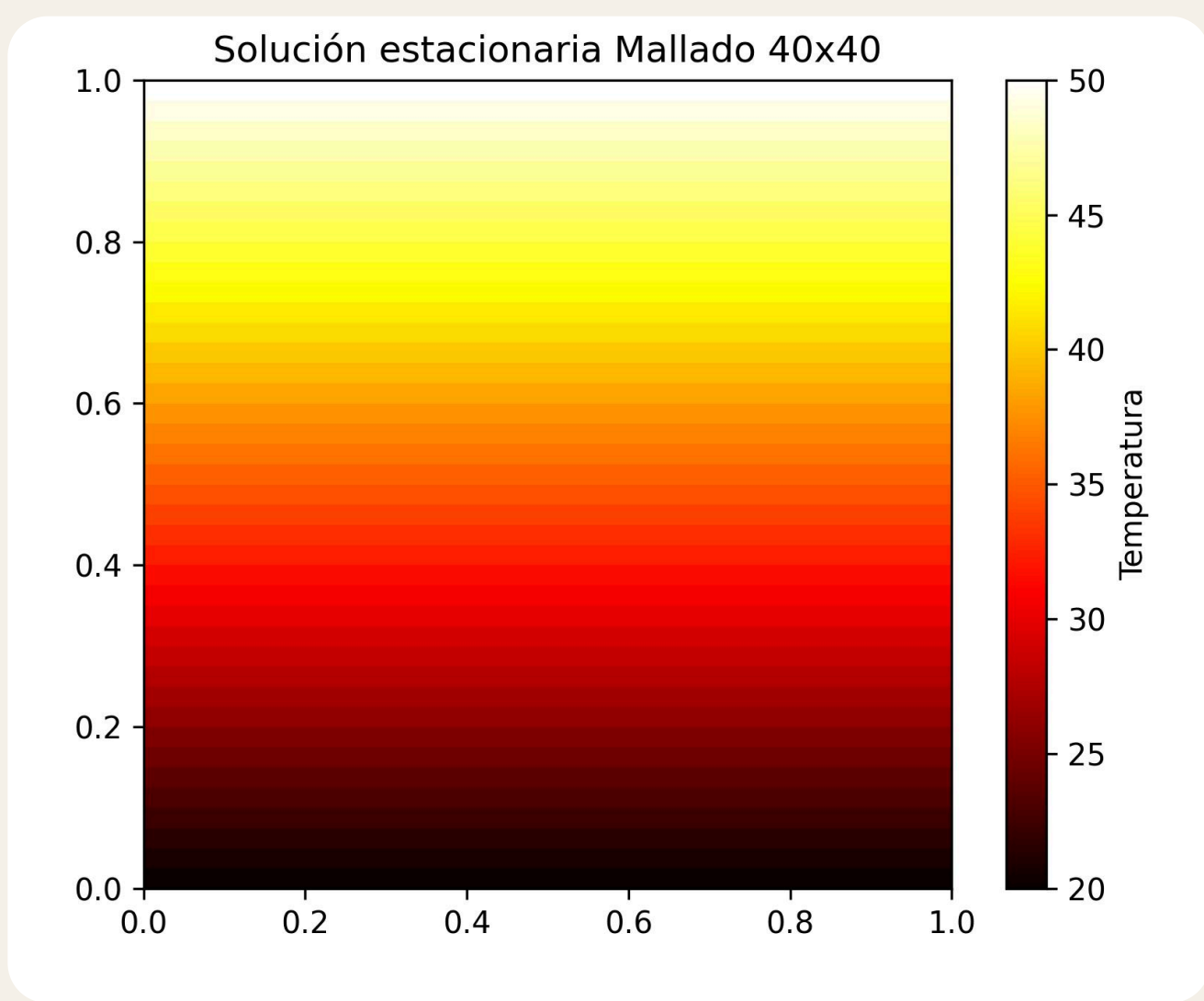
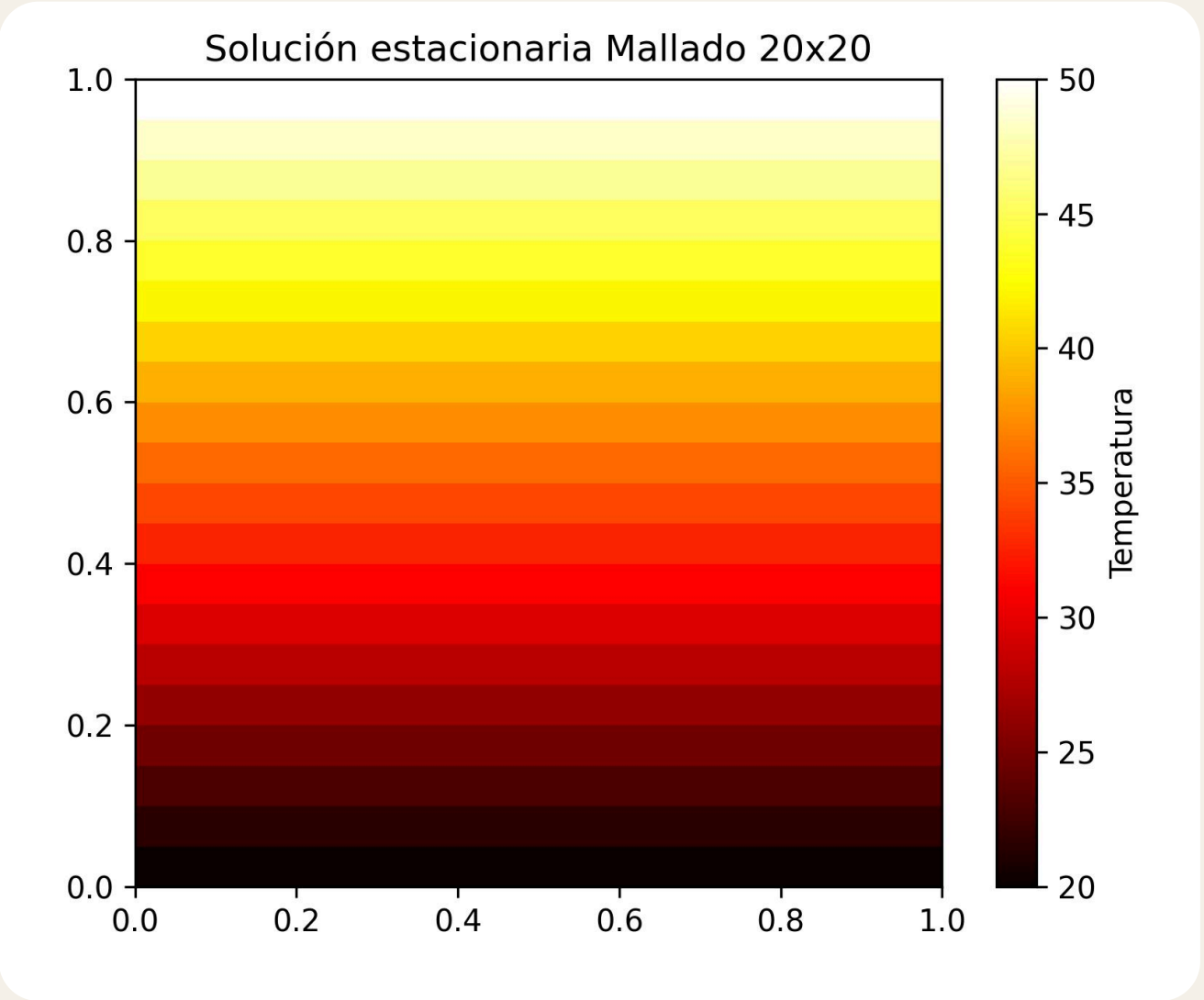
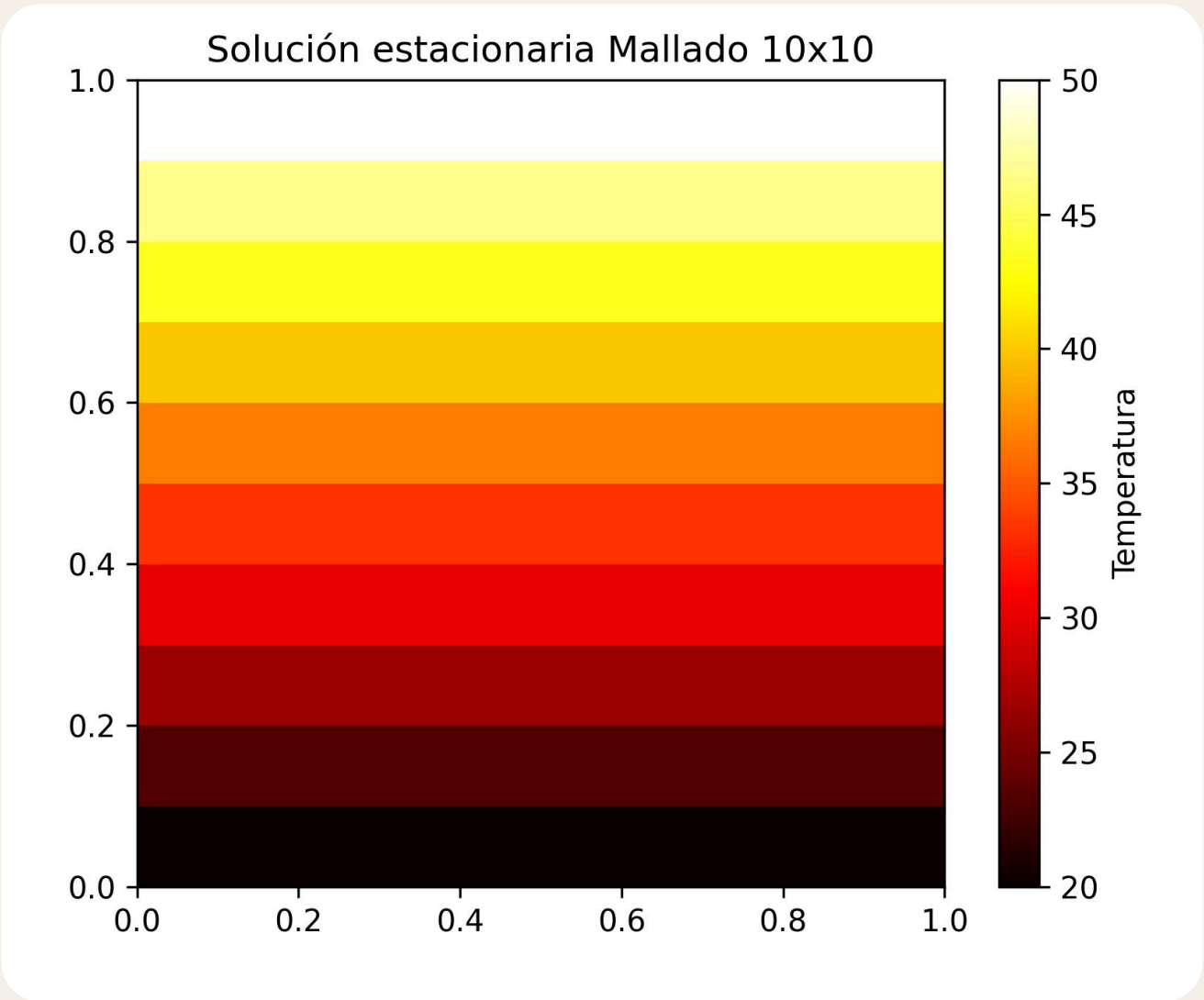
- Las soluciones con mallado 10×10, 20×20 y 40×40 muestran perfiles de temperatura muy similares.
- A mayor resolución, la transición entre temperaturas es más suave y precisa.

Mallado fino vs. grueso

- 10×10: rápido pero con gradientes poco precisos y transiciones bruscas.
- 20×20: buen equilibrio entre detalle visual y coste computacional.
- 40×40: solución más suave y precisa, con mejor definición de gradientes.

Robustez Numérica

- El método mantiene estabilidad y precisión al aumentar la densidad del mallado.
- Se confirma la convergencia hacia la solución teórica.
- **Importancia práctica:** Mallado fino mejora el detalle, pero incrementa el coste computacional.



Muchas Gracias!